

Master 1 - Semester 2
University Year 2024 - 2025
Second Session Exam
Info 408- Distributed System
Unauthorized documents
Grading scale / 100

July 2025

Duration 90 min

Exercise 1:

Write a Java program using UDP sockets to implement a "Multi-Client Game Server" that performs simple text operations and score tracking. The server accepts multiple clients, processes their commands, and maintains individual scores based on correct responses.

Server Side:

1. Listens on a specified UDP port for messages from multiple clients.
2. When a new client connects (detected by a "JOIN:username" message), the server:
 - Assigns a unique player ID (based on IP/port).
 - Initializes their score to 0.
 - Sends a welcome message and a random challenge (e.g., "Convert hello to uppercase").
3. Processes commands from clients:
 - ANSWER:<text>: Validates the client's answer.
 - > If correct, increases their score and sends "Correct! Your score is X".
 - > If incorrect, sends "Wrong answer, try again."
 - SCORE: Sends the current score to the client.
 - NEXT: Sends a new random challenge (e.g., "Reverse the string world").
 - EXIT: Removes the client from the active players list and sends a goodbye message.

Client Side:

1. Prompts the user for a username and sends "JOIN: username" to the server.
2. Receives a welcome message and a challenge (e.g., "Convert hello to uppercase").
3. The user can:
 - Send ANSWER:<text> to respond.
 - Send SCORE to check their score.
 - Send NEXT to get a new challenge.
 - Send EXIT to leave the game.

Exercise 2:

Write a Java program using TCP sockets to implement a "Multi-Client Online Banking Simulation Server".

Server side:

1. TCP Multi-threaded Server:
 - Listens for incoming client connections on a TCP port.
 - Spawns a separate thread for each client.
2. Account Management:
 - Each client logs in with a unique username and a predefined PIN (stored in the server's memory).
 - Each account has a balance initialized by the server.

3. Supported Commands:

- **BALANCE** : Returns the current balance.
- **DEPOSIT <amount>** : Adds the specified amount to the balance.
- **WITHDRAW <amount>**: Withdraws the specified amount if sufficient funds exist.
- **EXIT**: Logs the client out and closes the connection.

Client Side:

1. Connects to the server and logs in using username + PIN.
2. Displays a menu of operations (BALANCE, DEPOSIT, WITHDRAW, EXIT).
3. Sends commands to the server and displays server responses.

Exercise 3:

Write a program to design and implement a Distributed Library Management System using Java RMI, where multiple clients can connect to a remote server to manage and borrow books concurrently.

1- Define a remote interface LibraryService with the following methods:

- **addBook(String title, String author, int year)**: Adds a new book to the library.
- **borrowBook(String title, String username)**: Allows a user to borrow a book if it is available.
- **returnBook(String title, String username)**: Allows a user to return a borrowed book.

2- Server Implementation:

- Implement the LibraryService interface in a class LibraryServiceImpl.
- Register the LibraryServiceImpl object with the RMI registry under the name "LibraryServer".

3- Client Application:

- Create a client program that connects to the RMI registry to look up LibraryService.
- The client should present a menu to the user with the following operations:
 - > Add a new book.
 - > Borrow or return a book.